Spring form validation

학습목표

- spring form validation 구현
- 모델 클래스, DTO 클래스 개념

목 차

1. 배경지식 2
   1) DTO 클래스와 모델(model) 클래스 2
   2) spring form validation 3
2. form3 프로젝트 5
   1) 프로젝트 생성 5
   2) application.properties 7
   3) StudentEdit 모델 클래스 7
   4) Student Controller 구현 8
   5) student/edit.html 10
   6) student/list.html 12
   7) common.css 12
   8) 실행 13
3. validation 오류 문구 수정 14
   1) properties 에디터 인코딩 설정 14
   2) ValidationMessages.properties 파일 생성 15
   3) StudentEdit 모델 클래스 16
   4) 실행 17

1. 배경지식

1) DTO 클래스와 모델(model) 클래스

데이터를 채워서 전달하기 위한 클래스를 DTO 클래스라고 한다.

지금까지 학생 수정 화면의 입력폼에 입력된 데이터를 액션 메소드가 받을 때 Student DTO 클래스를 사용했다.

```
@PostMapping("edit")
public String edit(Model model, Student student) {
```

위와 같이 구현해도 틀린 것은 아니다.

그런데 좀 더 엄격하게 구현하자면,
입력 화면의 입력 폼에 입력된 데이터를 받기 위한 용도의 모델 클래스를 따로 구현하는 것이 좋다.

```
@PostMapping("edit")
public String edit(Model model, StudentEdit student) {
```

DTO 클래스는 주로 DB에서 조회한 정보를 채우는데 사용한다.

예:
학생 정보 DB 조회 - Student DTO 클래스

학생정보 수정 화면 - StudentEdit 모델 클래스
학생 등록 화면 - StudentCreate 모델 클래스

예:
회원 정보 DB 조회 - User DTO 클래스

로그인 화면 - UserLogin 모델 클래스
회원 가입 화면 - UserSignUp 모델 클래스
회원정보 수정 화면 - UserEdit 모델 클래스

요약:
DB에서 조회한 데이터를 채워서 전달하기 위한 용도 -> DTO 클래스
입력 화면의 입력 폼에 입력된 데이터를 전달 받아서 채우기 위한 용도 -> 모델 클래스

2) spring form validation

입력폼에 입력된 데이터를 spring이 자동 검사해 주는 기능이 spring form validation 이다.

```
@PostMapping("edit")
public String edit(Model model, Student student) {
    try {
        log.debug(student.toString());
        if (StringUtils.hasText(student.getStudentNo()) == false)
            throw new Exception("학번을 입력하세요");
        if (StringUtils.hasText(student.getName()) == false)
            throw new Exception("이름을 입력하세요");
        if (student.getDepartmentId() == 0)
            throw new Exception("학과를 선택하세요");
        if (StringUtils.hasText(student.getGender()) == false)
            throw new Exception("성별을 선택하세요");
        if (student.getYear() == 0)
            throw new Exception("학년을 선택하세요");

        // student 객체를 DB에 저장하는 구현 생략
        return "redirect:list";
```

지난 강의 예제에서는 입력된 데이터 에러 검사 기능을 직접 구현했다. (위 노란색 코드)

그런데 spring form validation 기능을 이용하면 다음과 같이 간단하게 구현할 수 있다.

```
@PostMapping("edit")
public String edit(Model model, @Valid StudentEdit student, BindingResult bindingResult) {
    try {
        log.debug(student.toString());
        if (bindingResult.hasErrors())
            throw new Exception("저장할 수 없습니다");

        // student 객체를 DB에 저장하는 구현 생략
        return "redirect:list";
```

위와 같이 구현하면, 입력된 데이터가 자동으로 검사된다. 편하다.
StudentEdit 모델 클래스를 사용하고 있음에 주목하자.

데이터 검사 규칙 어노테이션 (validation annotation)

데이터를 채울 `StudentEdit` 모델 클래스에 검사 규칙 어노테이션을 다음과 같이 달아주어야 한다.

```
@Data
public class StudentEdit {
    int id;

    @NotEmpty @NotBlank
    @Size(min=8, max=12)
    String studentNo;

    @NotEmpty @NotBlank
    @Size(min=2, max=20)
    String name;

    @Min(1)
    int departmentId;

    @NotEmpty @NotBlank
    @Pattern(regexp="남자|여자")
    String gender;

    boolean absense;

    @Min(1) @Max(4)
    int year;
}
```

spring form validation 어노테이션은 -> 모델 클래스의 속성에 붙인다

annotation	설명
@NotEmpty	값이 입력되었는지 검사한다
@NotBlank	공백만 입력된 것은 아닌지 검사한다
@Size(min=2, max=20)	입력된 문자열의 최소 크기 최대 크기를 검사한다
@Pattern(regexp="남자|여자")	입력된 문자열이 정규식에 일치하는지 검사한다
@Email	이메일 주소 문자열인지 검사한다
@Min(1)	입력된 값이 최소 1 이상인지 검사한다
@Max(10)	입력된 값이 최대 10 이하인지 검사한다

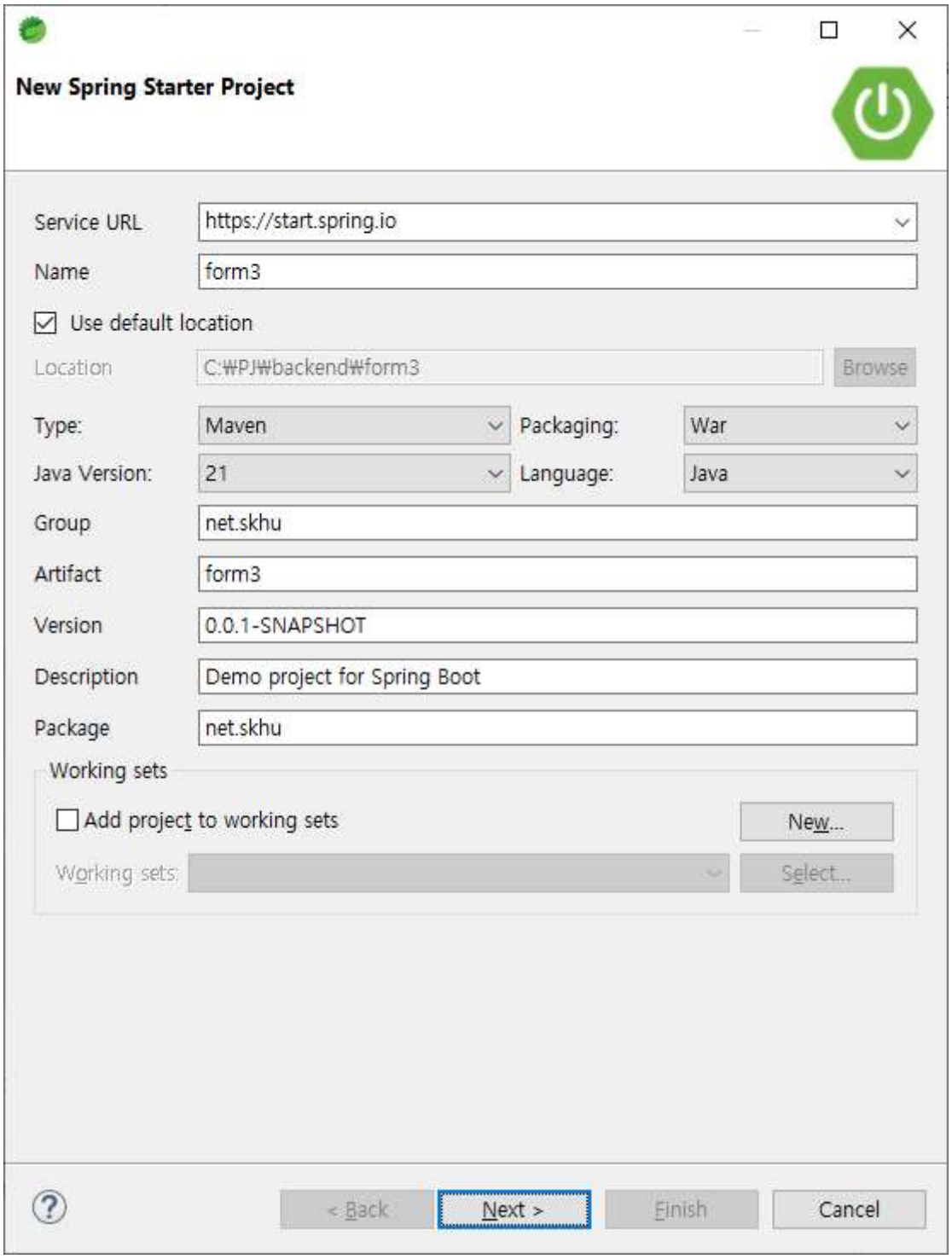
spring form validation 처리 과정

```
@PostMapping("edit")
public String edit(Model model, @Valid StudentEdit student, BindingResult bindingResult) {
    . . .
}
```

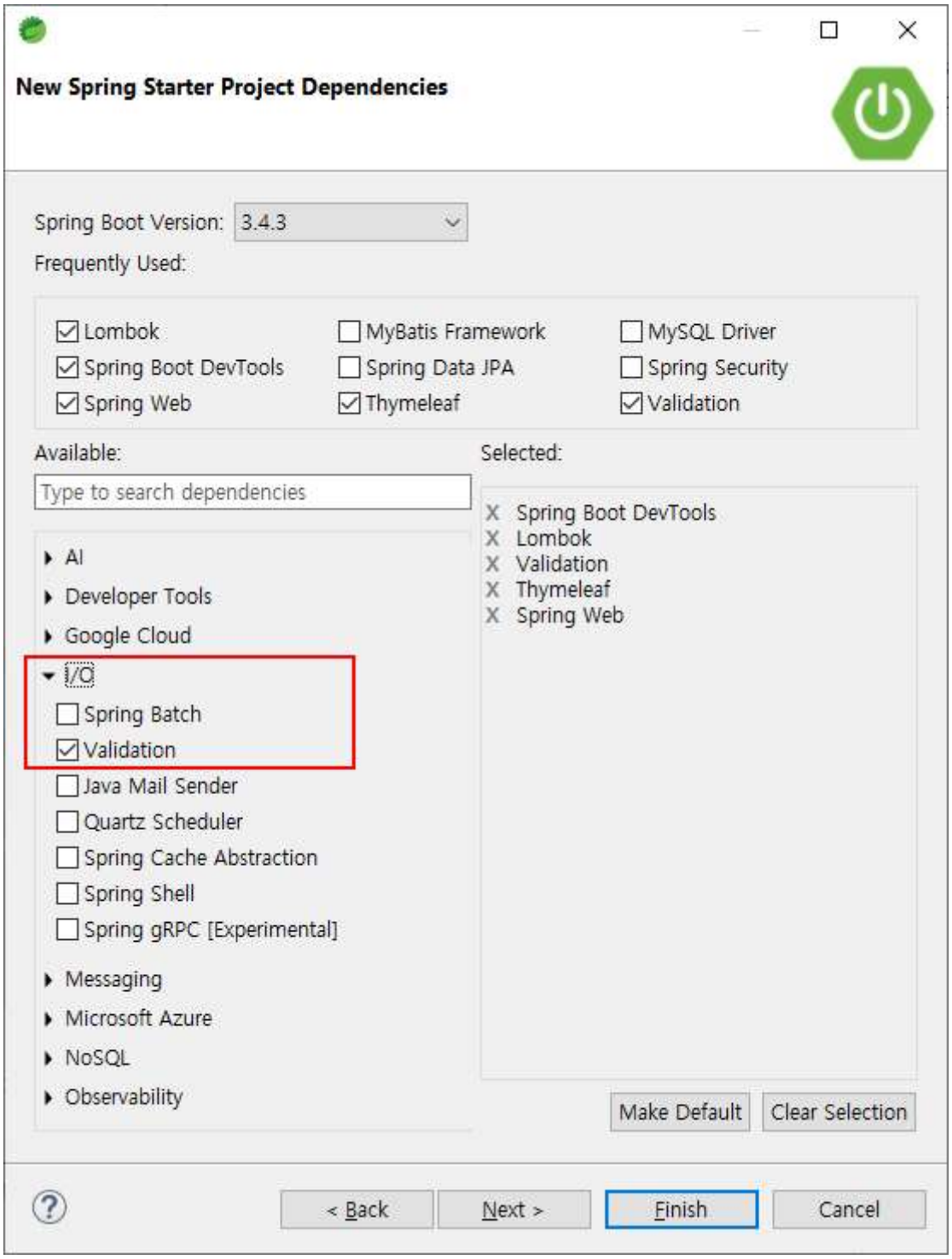
서버에 전달된 request parameter 데이터가 `StudentEdit` student 객체에 자동으로 채워진 후
validation annotation 규칙이 자동으로 검사된다.
검사 결과 오류가 있으면, 오류 메시지가 `BindingResult bindingResult` 객체에 자동으로 등록된다.

2. form3 프로젝트

1) 프로젝트 생성



Name	form3
Type	Maven
Java Version	21
Package	net.skhu



I/O 그룹 아래에서 Validation 항목을 체크한다.

2) application.properties

src/main/resources/application.properties

```
spring.application.name=form3
server.port=8088
logging.level.net.skhu=debug
```

3) StudentEdit 모델 클래스

src/main/java/net/skhu/model/StudentEdit.java

```java
package net.skhu.model;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class StudentEdit {
    int id;

    @NotEmpty @NotBlank
    @Size(min=8, max=12)
    String studentNo;

    @NotEmpty @NotBlank
    @Size(min=2, max=20)
    String name;

    @Min(1)
    int departmentId;

    @NotEmpty @NotBlank
    @Pattern(regexp="남자|여자")
    String gender;

    boolean absense;

    @Min(1) @Max(4)
    int year;
}
```

학생 수정 폼에서 입력한 데이터를 채우기 위한 모델 클래스이다.
입력된 데이터를 검증하기 위한 spring form validation annotation을 멤버 변수에 붙였다.

4) Student Controller 구현

src/main/java/net/skhu/controller/StudentController.java

```java
package net.skhu.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import jakarta.validation.Valid;
import lombok.extern.slf4j.Slf4j;
import net.skhu.model.StudentEdit;

@Controller
@RequestMapping("student")
@Slf4j
public class StudentController {

    @GetMapping("edit")
    public String edit(Model model) {
        StudentEdit student = new StudentEdit();
        student.setName("홍길동");
        log.debug(student.toString());
        model.addAttribute("studentEdit", student);
        return "student/edit";
    }

    @PostMapping("edit")
    public String edit(Model model, @Valid StudentEdit student, BindingResult bindingResult) {
        try {
            log.debug(student.toString());
            if (bindingResult.hasErrors())
                throw new Exception("저장할 수 없습니다");

            // student 객체를 DB에 저장하는 구현 생략
            return "redirect:list";
        }
        catch (Exception ex) {
            bindingResult.reject("", null, ex.getMessage());
            return "student/edit";
        }
    }

    @GetMapping("list")
    public String list() {
        // 학색 목록을 DB에서 조회해서 뷰에 전달하는 코드 생략
        return "student/list";
    }
}
```

학생 수정 화면 구현이기 때문에, StudentEdit 모델 클래스를 사용하였다

이름에 주의하자

클래스 이름	StudentEdit
model attribute 이름	studentEdit
변수 이름	student

액션 메소드의 파라미터가 객체이면
   그 객체는 자동으로 model.addAttribute 된다.
   이때 model attrubute 이름은 클래스 이름이다. (첫 글자만 소문자)

   변수 이름은 중요하지 않다.
   중요한 것은 클래스 이름과 model attribute 이름이다.

```java
@PostMapping("edit")
public String edit(Model model, @Valid StudentEdit student, BindingResult bindingResult) {
    try {

        // 입력된 데이터 검사 결과 오류가 있으면 exception을 throw 한다
        if (bindingResult.hasErrors())
            throw new Exception("저장할 수 없습니다");

        // student 객체를 DB에 저장하는 구현 생략
        return "redirect:list";
    }
    catch (Exception ex) {

        // exception이 발생했으면, exception message를 bindingResult 객체에 등록한다.
        // 이 bindingResult 객체에 입력 데이터 자동 검사 결과 오류 메시지들도 들어있다.
        // 이 bindingResult 객체도 자동으로 뷰에 전달된다.
        bindingResult.reject("", null, ex.getMessage());

        return "student/edit";
    }
}
```

```
bindingResult.reject("", null, ex.getMessage());
   bindResult 객체에 ex.getMessage() 에러 메시지를 등록한다.
     첫째 파라미터 "" 부분은 에러 메시지가 등록될 StudentEdit 객체의 속성명이다.
         즉 에러가 발생한 속성 이름이다.
     이 속성 이름 부분이 빈 문자열이면, 에러 메시지는 속성이 아니고 StudentEdit 객체 자체에 등록된다.

예:
   bindingResult.reject("studentNo", null, "학번이 중복됩니다.");
         // studentEdit.studentNo 속성에 에러 메시지 등록

   bindingResult.reject("", null, "저장할 수 없습니다.");
         // studentEdit 객체 자체에 에러 메시지 등록
```

5) student/edit.html

src/main/resources/templates/student/edit.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="/common.css" />
  <style>
    form { width: 500px; padding: 1em 3em; }
    button { margin-left: 1em; }
  </style>
</head>
<body>
<div class="container">
 <form method="post" th:object="${studentEdit}" class="shadow">
    <h1>학생 수정</h1>
    <div class="row">
      <label>학번:</label>
      <input type="text" th:field="*{studentNo}" />
      <div class="error" th:errors="*{studentNo}"></div>
    </div>
    <div class="row">
      <label>이름:</label>
      <input type="text" th:field="*{name}" />
      <div class="error" th:errors="*{name}"></div>
    </div>
    <div class="row">
      <label>학과:</label>
      <select th:field="*{departmentId}">
        <option value="0">학과를 선택하세요</option>
        <option value="1">소프트웨어공학과</option>
        <option value="2">컴퓨터공학과</option>
        <option value="3">정보통신공학과</option>
      </select>
      <div class="error" th:errors="*{departmentId}"></div>
    </div>
    <div class="row">
      <label>성별:</label>
      <label><input type="radio" th:field="*{gender}" value="남자" />남자</label>
      <label><input type="radio" th:field="*{gender}" value="여자" />여자</label>
      <div class="error" th:errors="*{gender}"></div>
    </div>
    <div class="row">
      <label>졸업:</label>
      <label><input type="checkbox" th:field="*{absense}" />휴학</label>
      <div class="error" th:errors="*{absense}"></div>
    </div>
    <div class="row">
      <label>학년:</label>
      <input type="number" th:field="*{year}" />
      <div class="error" th:errors="*{year}"></div>
    </div>
    <hr />
    <button type="submit" class="btn">저장</button>
    <span class="error" th:errors="${studentEdit}"></span>
  </form>
</div>
</body>
</html>
```

`${ }` 부분과 `*{ }` 부분을 구별하자.
하늘색 부분은 model attribute 이름이다. 예: `${studentEdit}`
연두색 부분은 객체의 속성이다. 예: `*{studentNo}`

```
<div class="error" th:errors="*{studentNo}"></div>
```
studentEdit 객체의 studentNo 속성 값에 대한 spring form validation 오류 메시지가
 태그 사이에 출력된다.
이 오류 메시지는 spring form validation 자동 검사 기능에 의해서
bindingResult 객체에 담겨서 뷰에 전달되었다.

```
<span class="error" th:errors="${studentEdit}"></span>
```
속성 값 오류 메시지가 아니고 studentEdit 객체에 대한 오류 메시지가
 태그 사이에 출력된다.
이 오류 메시지는 액션 메소드의 아래 코드에 의해서 bindingResult 객체에 담겨서 뷰에 전달되었다.
```
bindingResult.reject("", null, ex.getMessage());
```

6) student/list.html

src/main/resources/templates/student/list.html

```
<!DOCTYPE html>
<html lang="ko" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet" type="text/css" href="/common.css" />
  <style>
     div.container { width: 500px; padding: 2em; }
  </style>
</head>
<body>
<div class="container shadow">
   <h1>학생 목록</h1>
   <p>학생 목록 출력 생략</p>
   <p>저장되었습니다</p>
</div>
</body>
</html>
```

전과 동일

7) common.css

src/main/resources/static/common.css

```
div.container { width: 800px; margin: 10px auto; font-size: 11pt; }
.shadow { box-shadow: 2px 2px 5px gray; }

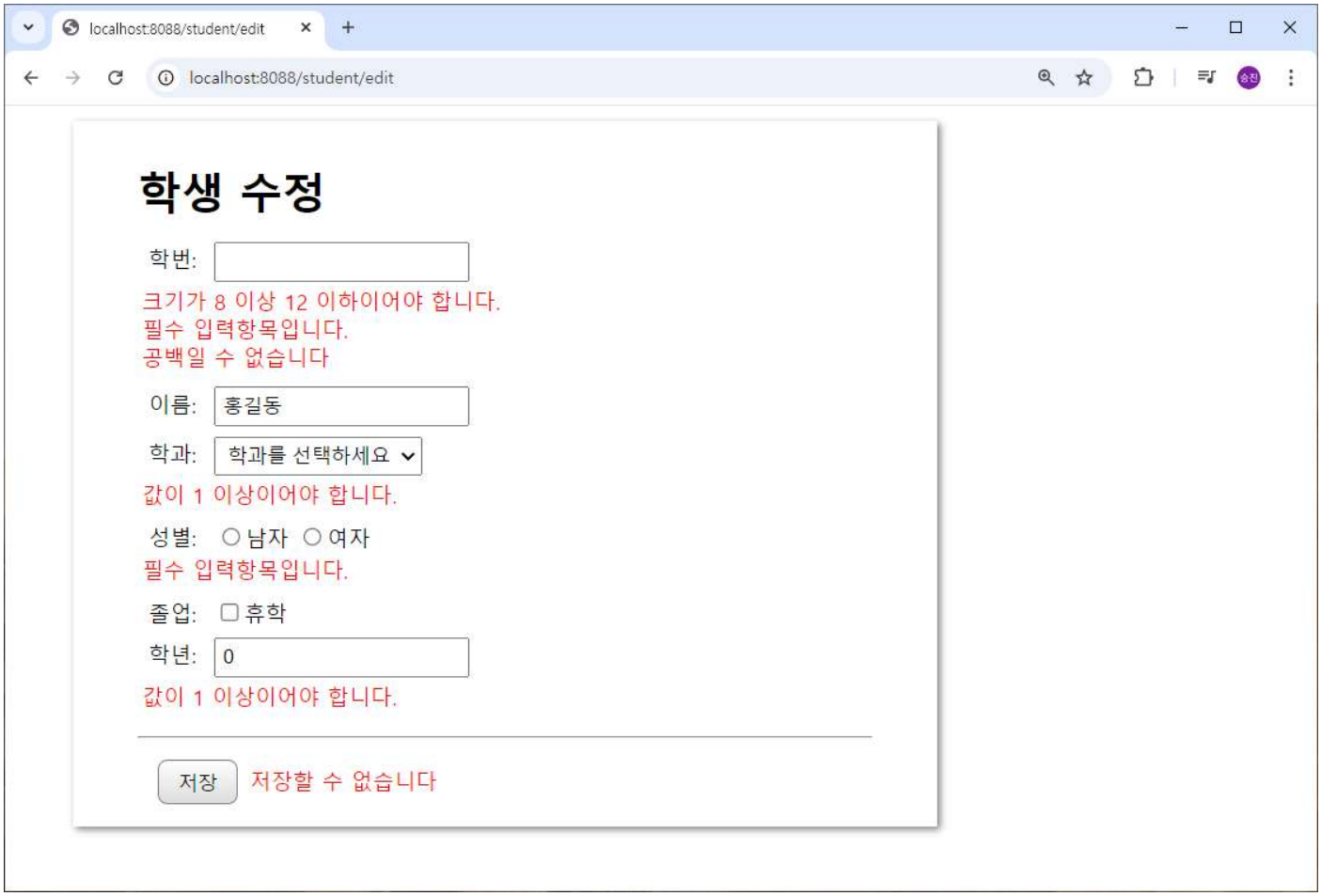
.btn { padding: 0.4em 1em; border: 1px solid gray;
       border-radius: 0.5em; background: linear-gradient(#fff, #ddd);
       text-decoration: none; color: black;
       display: inline-block; }
.btn:active {
      -ms-transform: translateY(2px);
      -webkit-transform: translateY(2px);
      transform: translateY(2px);
      background: #ccc; }

table.list { border-collapse: collapse; width: 100%; }
table.list td { padding: 4px; border: 1px solid gray; }
table.list th { padding: 4px; border: 1px solid gray;  background-color: #eee; }

div.row { margin-bottom: 0.8em; }
div.row > label:nth-child(1) { display: inline-block; min-width: 4em; text-align: right; }

input[type=text], input[type=number] { padding: 0.3em; }
select { padding: 0.3em; }
h1 { margin: 0.5em 0; }
hr { margin: 1em 0; }
.error { color: red; padding: 0.2em; margin-left: 1em; }
```

8) 실행



자동으로 출력된 오류 메시지 문구가 약간 맘에 들지 않는다.
오류 메시지 문구를 수정하자.

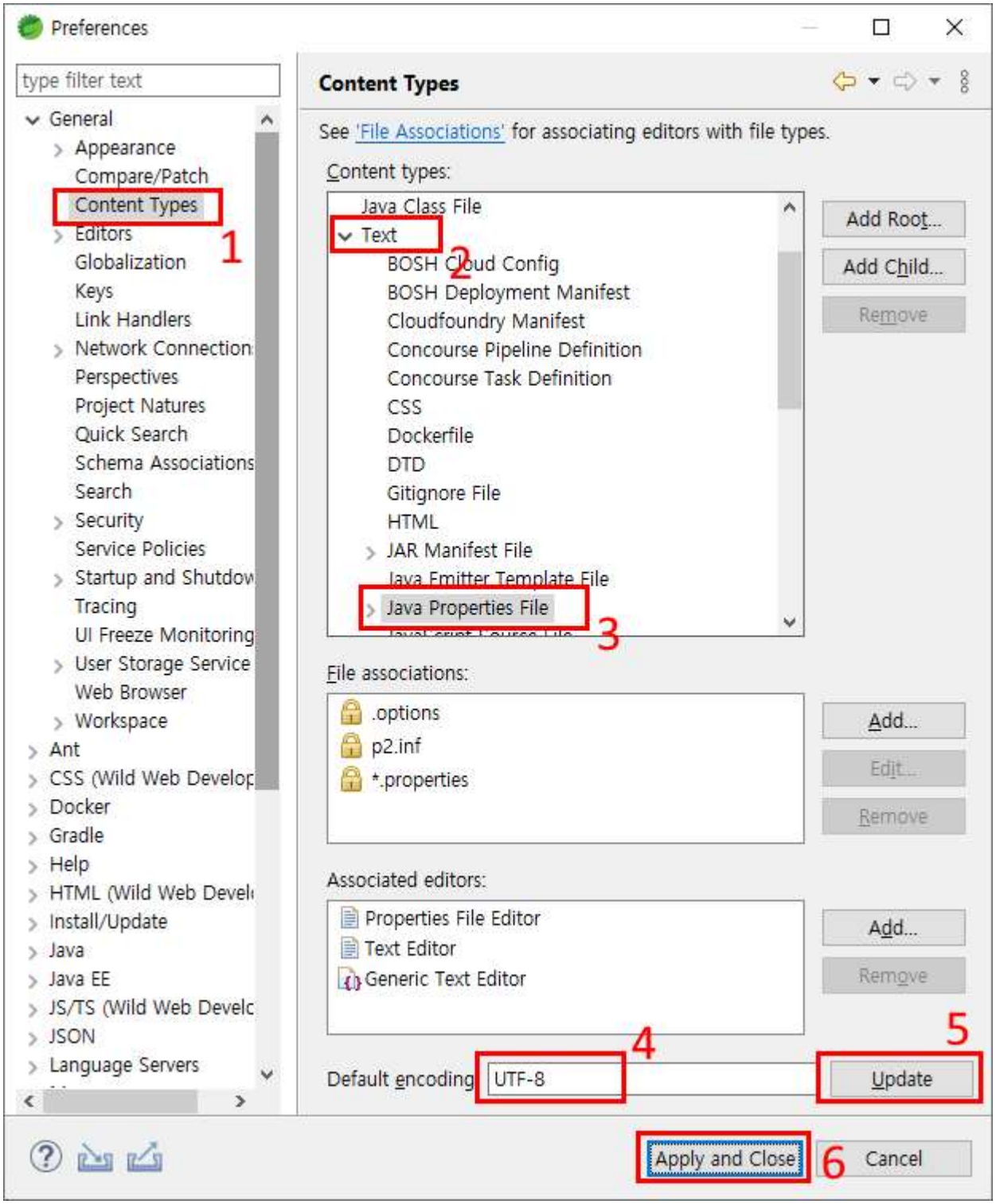
3. validation 오류 문구 수정

1) properties 에디터 인코딩 설정

기본 오류 문구를 등록할 ValidationMessages.properties 파일을 작성해야 한다.

그런데 Spring Tool Suite에서 *.properties 파일의 한글이 제대로 보이지 않는다.
*.properties 파일의 기본 인코딩이 UTF-8이 아니기 때문이다.
UTF-8로 변경해 주어야 한글이 제대로 보인다.

Window - Preferences 메뉴 클릭
General 항목 아래에 Content Types 항목



\

2) ValidationMessages.properties 파일 생성

src/main/resources/ValidationMessages.properties

```
jakarta.validation.constraints.Size.message=크기가 {min} 이상 {max} 이하이어야 합니다.
jakarta.validation.constraints.Email.message=이메일 주소가 바르지 않습니다.
jakarta.validation.constraints.NotEmpty.message=필수 입력항목입니다.
jakarta.validation.constraints.Max.message=값이 {value} 이하이어야 합니다.
jakarta.validation.constraints.Min.message=값이 {value} 이상이어야 합니다.
```

위와 같은 파일을 생성하면, spring form validation 오류 문구를 변경할 수 있다.

위 문구에서 {min} {max} 부분은
@Size 어노테이션에서 min 값과 max 값에 해당한다.

```
@Size(min=2, max=30)
String name;
```

위 문구에서 {value} 부분은, 어노테이션에 이름 없는 값이 한 개만 주어진 경우에 그 값이다.

```
@Min(10) @Max(50)
int amount;
```

위 구현과 아래 구현은 동일하다.

```
@Min(value=10) @Max(value=50)
int amount;
```

3) StudentEdit 모델 클래스

src/main/java/net/skhu/model/StudentEdit.java

```java
package net.skhu.model;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class StudentEdit {
    int id;

    @NotEmpty(message="학번을 입력하세요")
    @NotBlank
    @Size(min=8, max=12)
    String studentNo;

    @NotEmpty @NotBlank
    @Size(min=2, max=20)
    String name;

    @Min(value=1, message="학과를 선택하세요")
    int departmentId;

    @NotEmpty
    @Pattern(regexp="남자|여자")
    String gender;

    boolean absense;

    @Min(1) @Max(4)
    int year;
}
```

validation annotation에 오류 메시지를 각각 등록할 수도 있다.

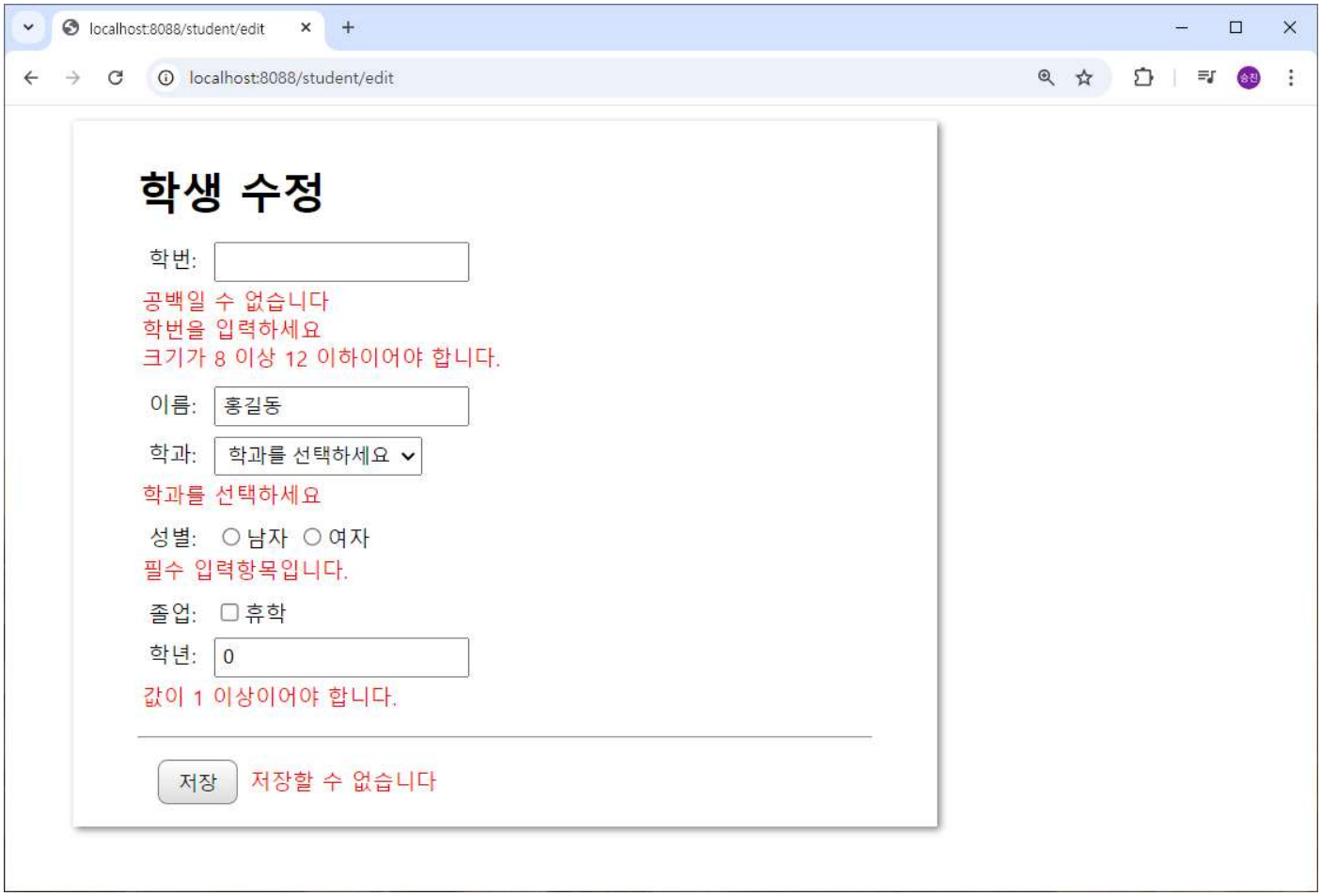
```
@NotEmpty
@NotEmpty(message="학번을 입력하세요")

@Min(1)
@Min(value=1, message="학과를 선택하세요")

@Pattern(regexp="남자|여자")
@Pattern(regexp="남자|여자", message="성별을 선택하세요")


@Size(min=2, max=20)
@Size(min=2, max=20, message="이름의 길이가 {min} 이상 {max} 이하이어야 합니다")
```

4) 실행


localhost:8088/student/edit
localhost:8088/student/edit
학생 수정
학번:
공백일 수 없습니다
학번을 입력하세요
크기가 8 이상 12 이하이어야 합니다.
이름:
홍길동
학과:
학과를 선택하세요
학과를 선택하세요
성별:
남자
여자
필수 입력항목입니다.
졸업:
휴학
학년:
0
값이 1 이상이어야 합니다.
저장
저장할 수 없습니다